SUPPLYED

SupplyED Project Audit Report

Security reliability data integrity and launch readiness

The audit identified fifteen issues across the web application. Immediate attention is required for vulnerable runtime dependencies, private data retained across account changes, the instant payout endpoint, and unfinished legal and account lifecycle flows.

Audit date 4 November 2026 **Workspace** supplyed-web **Branch** feat/stripe-reflow

Risk summary

URGENT	HIGH	MEDIUM	LOW
1	3	8	3

Prepared for the SupplyED project team

Executive summary

The application compiles and builds cleanly, and its role-based journeys cover a broad marketplace workflow. The main release risk is the interaction between security-sensitive dependencies, account-scoped browser state, payment mutations, incomplete lifecycle controls, and backend contracts that cannot be verified from this repository.

Priority decisions

• Patch the Next.js and Auth.js runtime before the next production deployment.

• Prevent private React Query data from surviving logout or account changes.

• Protect instant payouts with same-origin validation, atomic balance handling, and end-to-end idempotency.

• Complete legal review, privacy information, consent records, and the promised account closure workflow before launch.

Scope and validation

The review covered 485 workspace files and 96 application page or route entry points. It traced authentication, onboarding, teacher and institution journeys, administration, job posting, applications, invitations, bookings, attendance, messaging, notifications, documents, billing, payouts, deployment, and runtime behavior.

Validation	Result	Notes
ESLint	Passed	Full repository lint
TypeScript	Passed	No emit and incremental disabled
Production build	Passed	Next.js 16.2.9
Dependency audit	Failed	10 production vulnerabilities
Runtime smoke	Completed	Public page, redirect, health, headers, malformed bearer
Automated tests	Not available	No test files or test script

Audit boundaries

The external Nest backend, database schema, migrations, role enforcement, transaction boundaries, Stripe webhooks, scheduled jobs, storage cleanup, and backup controls are not present in this workspace. Findings that depend on those systems are marked as risks requiring backend verification rather than confirmed backend defects.

Findings register

The register provides a concise view of priority, subject, and verification status. Full evidence and recommended actions follow.

No	Priority	Finding	Status
1	Urgent	Vulnerable production dependencies expose reachable failures	Confirmed
2	High	Private query data can cross account boundaries	Confirmed
3	High	Cash out endpoint lacks origin protection and retry identity	Confirmed risk
4	High	Legal privacy consent and account closure flows are unfinished	Confirmed
5	Medium	Job publishing can create duplicate or orphaned jobs	Confirmed risk
6	Medium	My job lookup silently falls back to another school's public job	Confirmed
7	Medium	Schedules can overlap and double count hours and pay	Confirmed
8	Medium	Interrupted uploads can leave orphaned records and objects	Needs backend verification
9	Medium	Readiness reports healthy when the backend is unavailable	Runtime confirmed
10	Medium	Token refresh coordination breaks across multiple instances	Scaling risk
11	Medium	One rate limit blocks every unrelated feature	Confirmed
12	Medium	Social authentication advertises configurations that cannot work	Confirmed
13	Low	Malformed notification timestamps can crash notification views	Confirmed
14	Low	The application does not guarantee browser security headers	Deployment dependent
15	Low	Critical flows have no automated regression suite	Confirmed

Detailed findings

Each finding distinguishes the observed behavior from its likely effect and states the action needed to close the risk.

1 Vulnerable production dependencies expose reachable failures

URGENT | Security | Confirmed

What we found The production dependency audit reports ten vulnerabilities: four critical, four high, one moderate, and one low. Next.js 16.2.9 is affected by a Server Actions CPU exhaustion issue, and the project contains twenty Server Action modules. Auth.js also fails on malformed bearer tokens. A request with Authorization Bearer percent returned HTTP 500 and logged URIError URI malformed.

Why it matters Unauthenticated traffic can force error paths and, for the Next.js issue, consume enough CPU to block legitimate requests. Several additional advisories have reduced exposure because image optimization is disabled, use cache is absent, and the Auth.js email provider is not configured, but the two reproduced paths remain reachable.

Recommended action Upgrade Next.js outside every affected range, currently at least 16.4.1, and upgrade next-auth to beta 32 or newer with @auth/core 0.41.3 or newer. Update the remaining audited packages, replace or patch Quill's vulnerable HTML exporter, rebuild, rerun the dependency audit, and regression test authentication, OAuth, Server Actions, rich text, and uploads.

Evidence package-lock.json lines 52 to 58, 5359 to 5365, and 5412 to 5418; proxy.ts lines 43 to 50; runtime reproduction against the production build.

External references Next.js Server Actions advisory; Auth.js malformed bearer advisory

2 Private query data can cross account boundaries

HIGH | Privacy | Confirmed

What we found A single React Query client lives in the root layout. Private keys for conversations, notifications, jobs, settings, and billing do not include the current user ID. Logout navigates without cancelling or clearing the cache.

Why it matters A second person signing in within the same browser tab can receive the previous account's cached data. The data is fresh for thirty seconds and remains resident for up to five minutes, so it can appear immediately or while the new request is refetching.

Recommended action Include the immutable user ID in every private query key. Cancel and clear private queries before logout, on session expiry, and whenever the authenticated user changes. Add account switching tests for billing, messages, jobs, profiles, and notifications.

Evidence app/layout.tsx lines 79 to 91; lib/query/query-client.tsx lines 10 to 29; lib/query/keys.ts lines 25 to 66; components/organisms/RouteShell.tsx lines 125 to 129.

3 Cash out endpoint lacks origin protection and retry identity

HIGH | Payments | Confirmed risk

What we found An externally reachable POST route starts an instant payout without the project's requireSameOrigin guard. The UI uses a Server Action, but the duplicate API route remains directly callable. Both paths submit only amountPence and provide no stable idempotency key.

Why it matters A script on an untrusted sibling subdomain could issue an authenticated same-site request because SameSite cookies are scoped to the registrable domain rather than the exact origin. A lost response, reload, or second tab can also look like a new cash-out and may duplicate a payout if the missing backend does not enforce atomic idempotency.

Recommended action Remove the unused API route or enforce strict Origin, Referer, and Fetch Metadata validation. Require a persisted client-generated idempotency key, enforce uniqueness in the backend ledger, forward the key to Stripe, and perform balance deduction and payout creation atomically.

Evidence app/api/payments/payout-account/instant-payout/route.ts lines 8 to 15; features/payments/actions.ts lines 165 to 175; components/organisms/BillingPage.tsx lines 219 to 232.

External references OWASP Cross Site Request Forgery Prevention

4 Legal privacy consent and account closure flows are unfinished

HIGH | Compliance | Confirmed

What we found The published terms identify themselves as prototype copy requiring solicitor review. Signup checks a terms box, but registration sends only email, password, and role, so the backend receives no terms version or durable acceptance record. The terms promise that users can close their account from Settings, but the repository contains no closure action or interface.

Why it matters The product cannot demonstrate which terms a user accepted, and it publishes a user promise that the application does not fulfil. The short privacy section also omits several categories identified in current ICO guidance, including detailed lawful bases, controller contact details, retention periods, applicable rights, and complaint information.

Recommended action Obtain legal review, publish complete versioned terms and privacy information, and persist server-side acceptance records for email and social signup. Implement an account closure and data request workflow that respects outstanding bookings, financial records, safeguarding duties, and documented retention rules.

Evidence app/(public)/terms/page.tsx lines 15 to 18 and 118 to 123; features/auth/actions.ts lines 259 to 288; features/auth/backend.ts lines 356 to 370.

External references ICO Privacy Information Guidance

5 Job publishing can create duplicate or orphaned jobs

MEDIUM | Reliability | Confirmed risk

What we found Publishing performs two backend operations: create the job, then activate it. Recovery depends on a job ID held only in a component reference. If creation succeeds but the response is lost before the ID is stored, retrying sends another create request.

Why it matters Users can create duplicate postings, orphaned drafts, or active jobs that the interface reports as failed after a response loss or page reload.

Recommended action Make create and publish atomic or attach a persisted idempotency key to the creation attempt. Provide an attempt lookup and recovery endpoint, then test dropped responses, reloads, and concurrent tabs.

Evidence features/jobs/actions.ts lines 13 to 44; components/organisms/PostJobPage.tsx lines 210 to 228 and 366 to 385.

6 My job lookup silently falls back to another school's public job

MEDIUM | Authorization | Confirmed

What we found When scope mine cannot find an owned job, the route falls back to the public job endpoint. A school can direct-link to another school's public job and have it loaded into the edit flow.

Why it matters Backend ownership should still reject saving, so this is not an authorization bypass by itself. It does defeat the interface's ownership contract, exposes an edit experience for an unowned record, and produces misleading errors.

Recommended action Never fall back from the owned endpoint to public data. Return 403 or 404 and use a separate explicit public scope where required.

Evidence app/api/jobs/[id]/route.ts lines 6 to 21; features/jobs/queries.ts lines 47 to 57.

7 Schedules can overlap and double count hours and pay

MEDIUM | Data integrity | Confirmed

What we found Each weekday and exception is validated independently. A Monday shift from 23:00 to 02:00 can overlap a Tuesday shift from 01:00 to 03:00 without rejection. An add-hours exception can also overlap the recurring shift because the arrays are concatenated without merged validation.

Why it matters Preview hours and hourly pay sum both intervals, which can misstate planned work, expected pay, availability, and schedule conflicts.

Recommended action Expand the schedule into dated intervals, normalize overnight boundaries, sort the result, and reject overlaps after recurring shifts and exceptions are merged. Apply the same rule in the backend and test overnight, duplicate-add, boundary, and daylight-saving cases.

Evidence features/jobs/job-schedule.ts lines 15 to 35, 52 to 89, and 93 to 124; components/molecules/WorkScheduleEditor.tsx lines 37 to 45.

8 Interrupted uploads can leave orphaned records and objects

MEDIUM | Storage | Needs backend verification

What we found Conversation attachments create a backend record before uploading to storage. Failed storage writes have no compensating delete, and successful uploads become messages only in a later user action. Profile documents similarly create or reuse a row, upload an object, and then call a separate completion endpoint.

Why it matters Interrupted requests can leave pending database rows or unused storage objects unless the external backend runs a cleanup process that is not present in this workspace.

Recommended action Model uploads as expiring pending records, add an abort endpoint, make completion idempotent, and run a monitored time-to-live cleanup job with object-storage lifecycle rules.

Evidence app/api/conversations/[id]/attachments/route.ts lines 33 to 49; features/onboarding/documents.ts lines 350 to 401.

9 Readiness reports healthy when the backend is unavailable

MEDIUM | Operations | Runtime confirmed

What we found The health endpoint checks only whether environment values exist. It never contacts the backend. Both deployment health checks rely exclusively on this route, and runtime testing returned HTTP 200 with status ok.

Why it matters A wrong backend URL or complete backend outage leaves the frontend container healthy and receiving traffic, while users encounter broad failures.

Recommended action Separate liveness from readiness. Keep liveness process-local, but make readiness perform a short bounded backend health request and validate critical runtime configuration.

Evidence app/api/health/route.ts lines 5 to 13; deploy/docker-compose.main.yml lines 20 to 29; deploy/docker-compose.dev.yml lines 20 to 29.

10 Token refresh coordination breaks across multiple instances

MEDIUM | Authentication | Scaling risk

What we found Refresh deduplication uses a process-local map. Two replicas receiving parallel requests for the same rotating refresh token can both call the backend refresh endpoint.

Why it matters If the backend treats refresh tokens as one-use, one request can invalidate the other and sign the user out. The present Compose deployment is single-instance, but the design becomes unsafe when scaled or during overlapping deployments.

Recommended action Make backend token-family rotation idempotent or coordinate refreshes through a shared store or lock. Load test two application instances using the same session.

Evidence lib/server/token-refresh.ts lines 14 to 23 and 188 to 210.

11 One rate limit blocks every unrelated feature

MEDIUM | Availability | Confirmed

What we found A single browser-global until value records any HTTP 429 response. Every fetchJson call checks that value before sending a request.

Why it matters A rate limit from one sensitive operation can locally block jobs, messages, billing, notifications, and other unrelated reads for the entire cooldown period.

Recommended action Scope cooldowns by backend limiter bucket, endpoint class, and user. Continue safe reads unless the backend explicitly communicates a global limit.

Evidence lib/rate-limit.ts lines 5 to 30; lib/query/fetch-json.ts lines 27 to 55.

12 Social authentication advertises configurations that cannot work

MEDIUM | Authentication | Confirmed

What we found Google appears available whenever credentials exist, even if the separate backend OAuth flag is disabled. Microsoft can be configured as an Auth.js provider, but the backend explicitly rejects it and the interface has no Microsoft button.

Why it matters A deployment can show a Google button that always fails, while Microsoft configuration creates the appearance of support for a flow that users cannot complete.

Recommended action Derive interface availability from one capability check covering credentials, feature flag, and backend support. Complete Microsoft exchange and interface support or remove its advertised configuration until it is ready.

Evidence features/auth/social-auth.ts lines 7 to 11; features/auth/backend.ts lines 535 to 560; components/molecules/SocialAuthButtons.tsx lines 4 to 40; .github/workflows/dev-cicd.yml lines 100 to 108.

13 Malformed notification timestamps can crash notification views

LOW | User interface | Confirmed

What we found Any string is accepted as createdAt. An invalid value reaches Intl DateTimeFormat, which throws a RangeError.

Why it matters One malformed backend record can break the notification bell or the recent-notifications section instead of degrading gracefully.

Recommended action Validate and normalize ISO timestamps at the response boundary, provide a safe fallback, and test missing, invalid, future, and extreme values.

Evidence features/notifications/schemas.ts lines 13 to 30; components/organisms/NotificationBell.tsx lines 191 to 197; components/organisms/RecentNotifications.tsx lines 85 to 91.

14 The application does not guarantee browser security headers

LOW | Browser security | Deployment dependent

What we found Production-style responses lacked Content Security Policy, frame-ancestors or X-Frame-Options, Strict Transport Security, Referrer Policy, and Permissions Policy, while exposing X-Powered-By. An external reverse proxy may add these headers, but the repository and container do not.

Why it matters Clickjacking and other browser hardening depend on undocumented infrastructure and may be absent on a direct deployment path.

Recommended action Define the headers in Next.js or at the public ingress, disable the powered-by header, and create a Content Security Policy compatible with Stripe, OAuth, uploads, and the messaging socket. Verify the actual public response.

Evidence next.config.mjs lines 1 to 13 and production-style runtime response inspection.

External references OWASP Clickjacking Defense

15 Critical flows have no automated regression suite

LOW | Quality assurance | Confirmed

What we found No test, specification, or end-to-end files and no test script were found. Continuous integration installs dependencies, runs lint, and builds, but does not execute behavioral tests.

Why it matters Payments, authentication, permissions, schedules, retries, and multi-user isolation can regress while continuous integration remains green.

Recommended action Add unit tests for boundary logic, backend-contract integration tests, and browser journeys for every role. Prioritize account switching, payout retries, job response loss, schedule overlap, upload interruption, token refresh, and authorization failures.

Evidence package.json lines 5 to 10; .github/workflows/dev-cicd.yml lines 17 to 41; repository-wide test-file search returned no test files.

Recommended implementation order

The sequence below reduces exposure first, then closes reliability and launch-readiness gaps. Timing assumes the external backend team is available for the payment, token, upload, and transaction work.

Window	Focus	Findings	Exit condition
Before deployment	Runtime security	1	Dependency audit no longer reports the exposed Next.js and Auth.js advisories
First 7 days	Privacy and payments	2 to 4	Account switching is isolated, payouts are protected and idempotent, and legal owners approve the launch plan
Weeks 2 to 4	Data and reliability	5 to 9	Mutation retries, ownership lookup, schedules, upload cleanup, and readiness are covered by tests
Weeks 4 to 6	Scale and experience	10 to 14	Multi-instance refresh, limiter scoping, social auth, timestamp safety, and public headers are verified
Ongoing	Regression protection	15	Role-based browser tests and backend-contract tests run in continuous integration

Verification still required

• Inspect the Nest backend, database schema, migrations, authorization rules, idempotency records, Stripe webhook handling, and transaction boundaries.

• Run complete journeys with real teacher, institution, and administrator accounts plus Stripe, OAuth, email, socket, and object-storage services.

• Test at least two application instances to validate refresh-token rotation, caches, and concurrent mutation handling.

• Inspect the public ingress for TLS, security headers, rate limits, trusted proxy behavior, monitoring, and readiness routing.

• Obtain qualified legal and operational approval for terms, privacy information, retention rules, safeguarding statements, contractor status, and account closure.

Positive controls observed

The project includes useful controls: same-origin enforcement on document uploads and public approval actions, strict upload checks, sandboxed document previews, proposal sanitization, server-side route checks, non-root production containers, restart and log-rotation policies, and secret redaction in authentication debug output. These controls reduce risk but do not remove the findings in this report.